Technical Details for Full Proposal

1. Origin of the proposal

Over the past decade, India has witnessed an unprecedented surge in demand for skilled software developers. According to the Stack Overflow Developer Survey 2023[1], only 42% of Indian developers report confidence in their core programming abilities. The World Economic Forum projects over 90 million digital jobs in India by 2025, yet only about 47% of engineering graduates are deemed employable for IT roles (NASSCOM 2022[2]), exposing a critical skills gap. Initiatives like Digital India and Skill India make strong programming skills essential for economic growth, social mobility, and inclusive access to well-paid, innovative roles in the global digital economy. Yet, conventional methods of training in computer programming in Indian schools fail to meet such expectations.

Large class sizes, low instructor bandwidth, and a single-size fit all curriculum lead to disparate learning outcomes. Students having trouble with core concepts do not get enough personal attention, which results in frustration, disaffection, and high dropout levels. Additionally, human TAs are usually tasked with mundane tasks, grading homework, responding to repetitive questions, and giving elementary feedback, which leaves no time for abstract, conceptual mentorship. Consequently, numerous graduates come out with theoretical information but are short on the practical skills and self-assurance needed by employers.

As a counter to these difficulties, various e-learning websites and intelligent tutoring systems have been proposed. Massive Open Online Courses (MOOCs) like Coursera and edX provide programming tracks, but their broad content tends not to fill learners' specific gaps. Automated assessment tools like CodeRunner[3] and HackerRank[4] offer immediate correctness checking but provide little interpretive feedback and no remediation at the individual level. Adaptive learning technologies, though promising, tend to leverage rule-based engines that are not scalable to the expressiveness of programming concepts and do not have good natural language support for subtle student questions. Lastly, most in-house academic solutions are not fully utilized because they are too expensive to implement, have a costly learning curve, and do not integrate well with existing institutional processes. The aiTA system, proposed in this work, is intended to overcome these limitations by integrating modular AI agents into an integrated, adaptive environment for the Indian education scenario.

The proposed aiTA system aims to transform programming education through a multi-agent, modular architecture. A User Profiling Agent repeatedly tunes learner models, providing personalized routes to a Content Recommendation Agent based on collaborative filtering and reinforcement learning. An Interactive Content Player Agent, supported by fine-tuned small language models, provides natural language explanations as well as immediate error detection. The Code Evaluation and Semantic Feedback Agents review code correctness and provide deep, concept-based feedback. Participation is enhanced through a Gamification Engine, while an Assessment Generation Agent generates adaptive tests. Built to integrate smoothly with LMS and to deploy microservices, aiTA offers strong fault isolation, thus enabling scalability and availability. In general, this proposal is targeting the nexus of India's imperative to skill millions of prospective technologists, the established learning advantage of adaptive, individualized tutoring, and the readiness of AI methods, specifically fine-tuning of small language models and reinforcement learning, which can now be orchestrated into a workable, composable pedagogical assistant. By solving pedagogical, technological, and operational issues together in an integrated framework, aiTA vows to revolutionize programming education in India, empowering students, maximizing teacher effort, and filling the employability gap that has long held India back from achieving its digital workforce dreams.

2. State of Art Review in the Proposed Subject

2.1 Review the status and highlight the significance of the proposed work:

1. Scientific rationale of the proposed work being taken up for study

The integration of Artificial Intelligence (AI) into educational settings has led to the emergence of Automated Teaching Assistants (ATAs), which significantly enhance both instructional delivery and learner support. Large-

---

[1] https://survey.stackoverflow.co/2023/

2https://www.financialexpress.com/jobs-career/education-engineering-dreams-employment-crisis-only-10-of-indias-1-5-million-graduates-set-to-find-jobs-says-report-3622825/

3 https://coderunnerapp.com/

4 https://www.hackerrank.com/

scale classrooms often struggle to provide personalized learning experiences, timely feedback, and individualized resources. AI-powered adaptive systems address these challenges by continuously analyzing a wide spectrum of student data, including academic performance, engagement patterns, study habits, and interests, collected from both online and offline sources.

One such system integrates GPT-3.5 within a progressive web application to offer intelligent support through text embeddings, intent and entity recognition, and a comprehensive knowledge base [1]. It delivers context-aware flashcards, quizzes, and real-time code execution environments. The system automates formative assessments and integrates an audience response module that captures live student feedback. These insights feed into analytics dashboards, providing valuable feedback for students, instructors, and teaching assistants. Another solution utilizes large language models (LLMs) to generate instructional content, such as reading quizzes, programming exercises, and learning objectives, with educators overseeing and refining the outputs to ensure pedagogical accuracy and relevance [2]. Specialized frameworks like CourseAssist [3] employ Retrieval-Augmented Generation (RAG) to ground AI responses in course-specific materials. These systems decompose complex student queries into sub-questions, delivering structured hints aligned with learning objectives. Similarly, the GenMentor multi-agent framework [4] maps learner-defined goals to required skills using chain-of-thought reasoning. It identifies individual skill gaps, creates dynamic learner profiles, and adapts personalized learning paths in real-time.

Common strengths across these platforms include seamless integration of LLMs and natural language processing (NLP), real-time assistance, and demonstrable improvements in student engagement, learning outcomes, and instructional scalability. However, notable limitations persist. These include a heavy dependence on data quality and completeness, technical complexity in integrating with existing Learning Management Systems (LMS), the ongoing challenge of balancing scalability with personalization, and ethical concerns regarding privacy, academic integrity, and algorithmic bias.

In the domain of programming education, the programmable Teaching Assistant [5] (pTA) automates the evaluation of student code submissions. Students submit containerized code to repositories like GitLab or GitHub. pTA retrieves, builds, and executes the code within isolated environments to ensure security and consistency. It conducts functional validation through network protocol tests and benchmarks performance using the Java Microbenchmark Harness (JMH). Deployed over multiple semesters, pTA has been shown to reduce instructor workload by over 75%, double course capacity, and enable uninterrupted online teaching during pandemic disruptions. To overcome the challenge of limited annotated data, ProtoTransformer [6] applies a meta-learning strategy for few-shot feedback classification. It uses Transformer-based encoders (inspired by RoBERTa) to embed tokenized student code and maps feedback categories (e.g., “syntax error,” “missing variable”) into a vector space where Prototypical Networks cluster examples. This approach enables accurate classification of new submissions with minimal labeled data.

Complementing these efforts is CodeAid [7], a client-server web application that leverages GPT-3.5 Turbo while upholding academic integrity. It offers five key support features: conceptual explanations, contextual code queries, annotated fix suggestions (without revealing solutions), line-by-line commentary, and high-level pseudocode. Its architecture employs prompt engineering with few-shot examples to maintain educational structure and pedagogical alignment. The tool includes pseudocode scaffolding, streaming LLM responses, static C library documentation, and interactive follow-ups. In a deployment with over 700 students in a C programming course, CodeAid improved response correctness from 74% to 87% and received strong student satisfaction ratings. Further, CodeAid integrates a subgoal-based learning model, where students define subgoals, implement solutions with AI-generated hints, and then compose complete programs. This approach promotes computational thinking while limiting direct solution exposure, thereby supporting both learning and academic integrity.

2. Recent work done in the chosen area by other groups in India or abroad

AI-enhanced ATAs are also being used for automatic code generation and explanation [8]. Using OpenAI’s Codex[5], entire programming tasks, solutions, test cases, and line-by-line explanations can be generated from structured prompt templates. Human evaluators and automated tests have verified that about 75% of these AI-generated problems are pedagogically sensible and around 82% are novel, demonstrating the scalability of such content generation, albeit requiring expert vetting to ensure accuracy and alignment with course goals. In terms of automated assessment, LLM-based graders such as GPT-4-powered “LLM TAs” are being used to align

[5] https://openai.com/codex/

prompts with course rubrics, enabling immediate scoring and self-assessment. However, they require safeguards against formatting inconsistencies and over-reliance on prompt tuning. Hybrid assessment models, which combine the strengths of human TAs and automated systems, strike a favorable balance between grading efficiency, accuracy, and learner satisfaction.

Virtual Teaching Assistants (VTAs) equipped with conversational capabilities interpret student intent, retain dialogue context, and use Retrieval-Augmented Generation (RAG) to ground responses in relevant course content. Deployed in MOOCs and LMS platforms, these VTAs respond to both academic and administrative queries, improving accessibility, supporting student autonomy, and enhancing engagement. Taken together, these ATAs and feedback automation systems represent a promising technological shift in education. They demonstrate high technical feasibility, leveraging scalable architectures such as microservices, containerization, and cloud deployment. These solutions integrate well with existing educational infrastructures and are particularly impactful in programming education, where real-time feedback, debugging support, and personalized instruction are critical. Empirical evaluations report significant reductions in instructor workload, heightened student engagement, and measurable learning gains.

3. The gaps in current research

Nonetheless, to realize the full potential of these systems at scale, further attention is required to address persistent issues: ensuring high-quality and diverse training data, navigating LMS integration challenges, maintaining ethical safeguards, and resolving the trade-off between mass deployment and individual customization. Systems such as Retrieval-Augmented Tutors [3] and CodeHelp [9] emphasize subgoal workflows and hint-driven guidance to nurture problem-solving. They retrieve relevant resources, decompose complex queries, and restrict direct solution delivery, fostering student planning, decomposition, and reflection while preserving academic integrity. Collectively, these works demonstrate the feasibility and impact of integrating LLMs, meta-learning, containerization, and RAG into teaching assistants that scale personalized support, automate routine tasks, and promote active learning. Remaining challenges include data quality, LMS integration, balancing personalization with scalability, and ensuring ethical safeguards gaps the proposed aiTA system aims to address in the Indian programming-education context.

2.2 Importance of the proposed project in terms of impact and innovation

The proposed aiTA system aims to advance computer programming education, particularly in the Indian context. Through filling the limitations in personalized, scalable, and interpretable AI-founded learning, aiTA transcends state-of-the-art programming education in a number of significant aspects.

1. Bridging the Digital Divide in Computer Programming Education: Despite India's vast pool of engineering skills, huge inequalities remain in the availability of high-quality programming training, particularly in tier-2 and tier-3 institutions, where teacher bandwidth is a constraint and class sizes are large. Conventional e-learning solutions provide one-size-fits-all content that is unable to respond to individual student requirements, isolating struggling learners. aiTA explicitly addresses this divide by providing a multi-agent, modular architecture that individualizes the journey for every student. The User Profiling Agent models each learner’s strengths, weaknesses, and interests to avoid one-size-fits-all recommendations, while the Content Recommendation Agent fine-tunes resources in real time to fill knowledge gaps. Combined with personalized dashboards and adaptive learning paths, aiTA delivers individualized instruction.

2. Application of Small Language Models (SLMs) and Multi-Agent Framework: AI tutors based on existing approaches mostly utilise monolithic models or rule-based engines. Compared to that, aiTA innovates a multi-agent system of collaboration between fine-tuned SLM-based agents, optimized for low-latency inference, serving up real-time code explanations, brainstorming ideas, and error identification. A System Orchestration Agent directs tasks-invoking the Code Evaluation Agent, the Semantic Feedback Agent, and the Assessment Generation Agent. This agentic design increases scalability, maintainability, and future extendibility, with the ability to introduce new agents.

3. Explainability and Trust in AI-Driven Feedback: A significant barrier to the adoption of AI within education is the lack of transparency. The proposed aiTA revolutionizes standard explainers by using an LLM-powered Answering Agent that grounds these explanations in natural language so that learners can ask "Why this feedback?" and get explicit reasons. By making explainability a prime concern, aiTA guarantees that students and teachers can check, critique, and refine learning.

4. Cost-Effective, Generative Assessment and Gamification: Existing systems tend to emphasize static question banks. aiTA's Content-Generation Module applies generative AI combined with distilled SLMs to automatically generate quizzes and practice exercises specific to each student's profile and create dynamic explanations that adjust to a student's incorrect answers, reinforcing concepts that were missed. Combined with a Gamification Engine Agent that grants points, badges, and leaderboard rankings, aiTA maintains interest through intrinsic and extrinsic motivation. This real-time, adaptive gamification is a new pedagogical innovation that combines adaptive learning with behavioral psychology.

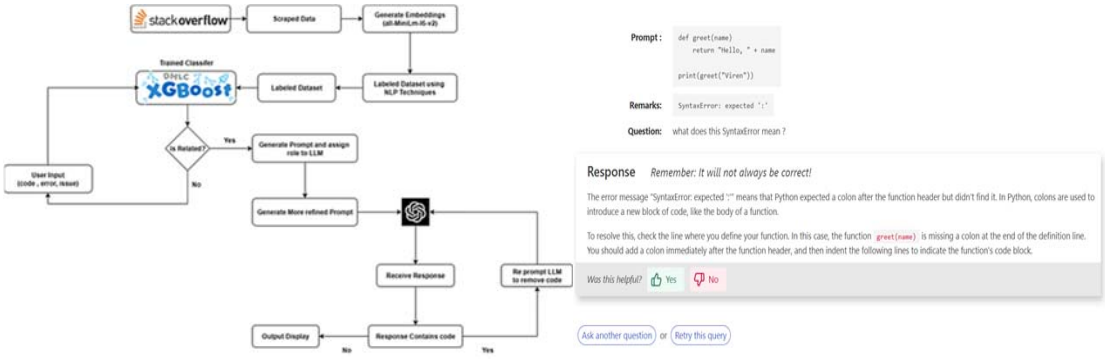
5. Comprehensive Validation in Real-World Indian Context: Most existing systems are tested using benchmark datasets, whereas aiTA commits to conducting controlled trials with diverse Indian institutions to mimic real-world resource limitations and linguistic diversity. By assessing learning efficiency gains, error-reduction rates, and changes in student programming confidence, under IRB-approved quasi-experimental designs, aiTA will produce robust empirical evidence of its educational effectiveness.

6. Alignment with National Priorities and Scalability: aiTA is well-placed to drive India's NEP 2020 vision for tech-enabled, student-focused education and the Digital India mission to digitize public services through digital innovation. By framing aiTA as an open-source, containerized solution that can be deployed on-premises or in cloud infrastructures, it can grow quickly.

2.3 Proof of concept can be shared with appropriate preliminary data

The CodeHelp [9] provides a platform for students to submit their code, error messages, and issue descriptions, then uses general-purpose Large Language Models (LLMs) to diagnose and suggest fixes. However, by treating code, errors, and issues as a single prompt token, it can't verify that the student-reported problem matches the actual error. This mismatch can lead to irrelevant advice, higher cognitive load, and unnecessary LLM calls. In our preliminary work, we identified the correlation between the errors and issues given by the students for a source code. The objective was to reduce the size of the token and simultaneously reduce the cost of executing tokens in the LLMs. The approach consisted of three steps: (a) creation of error-issues and source code dataset, (b) applying NLP techniques for pre-processing and labelling, (c) model generation.

Dataset Creation: We have collected a dataset from StackOverflow using its API by scraping questions from each page. Each question consists of the code, error, and the issue (the user asked the question by specifying the code and error). We have collected **35,000 samples** for each of the three **features**: code, error, and issue.



(a) (b)

Fig. 1: (a) Overview of the approach analyzing the correlation between the error and issues for a source code, (b) Developed Prototype

Labelling and Pre-processing: After creating the dataset, we pre-processed it by removing the missing values (rows or columns with no values). We removed 2234 rows with missing values and created the final pre-processed dataset with 32,776 Rows. After pre-processing, we need the labelled dataset for supervised learning for the model. So, we labelled it as 0 or 1 using NLP techniques. Here, the label 0 indicates that errors and issues have no correlation, and 1 indicates correlation.

Model Generation: Initially, we selected Logistic Regression (LR) because of its simplicity and interpretability. However, LR fails in capturing complex relationships between error and issue; hence, we choose XGBoost due to its capabilities of handling non-linear interactions between features (error and issues). We use Sentence Transformers for embeddings. We did hyper parameter tuning of the XGBoost model (e.g., adjusting the number of estimators, learning rate, and tree depth) to optimize performance and prevent overfitting. We got an accuracy

of 87.8% in the XGBoost model 87.8%, whereas in LR, we got an accuracy of around 85%. However, there is not much difference in terms of accuracy, but XGBoost, along with the sentence transformer, works well with textual data and captures the complex non-linear relationships. Our future plan is to develop an approach for code analysis agents that examine errors and issues and generate ways to solve the issue, similar to how users post their answers in Stack Overflow. The code analysis agent will facilitate the solution to the issues users are facing with the programming problems.

2.4 If the project is location specific, basis for selection of location be highlighted:

Not Applicable

3. Work plan

Our methodology broadly focuses on two different aspects: (a) development of an intelligent automated Teaching Assistant (TA) system named *aiTA* and (b) evaluation of the proposed framework by conducting experimental analysis with the students, instructors, and human TAs for its effectiveness, efficacy, usability, and impact. The proposed system makes use of natural language processing, fine-tuned language models, code analysis, gamification-based learning, and customised content recommendation systems to support students throughout their programming learning journey. Fig. 2 gives an overview of the proposed aiTA framework.

3.1 Methodology:

Work Item-1: Development of an Intelligent Automated TA framework:

The development of the proposed intelligent automated TA system involves building specialised agents to perform various tasks together in the aiTA system. The specialised AI agents intend to utilise AI techniques to achieve specific goals.

Step 1 (User Profiling Agent): The working of the aiTA system is initiated by a user logging in and providing basic details, which activates the *User Profiling Agent*. An agentic AI-based *User Profiling Agent* collects data to build a comprehensive view of individual users, their learning behavior, demographic profile, personal interests, and experiences. This profiling is refined using a multi-modal approach: explicit input via direct user settings, analysis of behavior via examination of interactions, e.g., view history, navigation behavior, and error types, and contextual data. The agent then applies machine learning techniques, e.g., clustering, classification, and natural language processing, to construct all of this into a dynamic user profile so that it can identify interests, set up skill levels, and even infer user goal activity. The ethical concerns of such profiling are of utmost importance. Since the User Profiling Agent takes distinct user data as input to create the user profile, privacy and data protection need to be ensured by adopting protection measures, informed consent, data minimization, explainability, and transparency The user should be informed that what kind of data is being gathered, how their profile is being constructed, and be able to view and correct their data. Also, we address algorithmic bias to ensure fair, non-discriminatory outcomes, while opt-out options and human oversight preserve user autonomy.

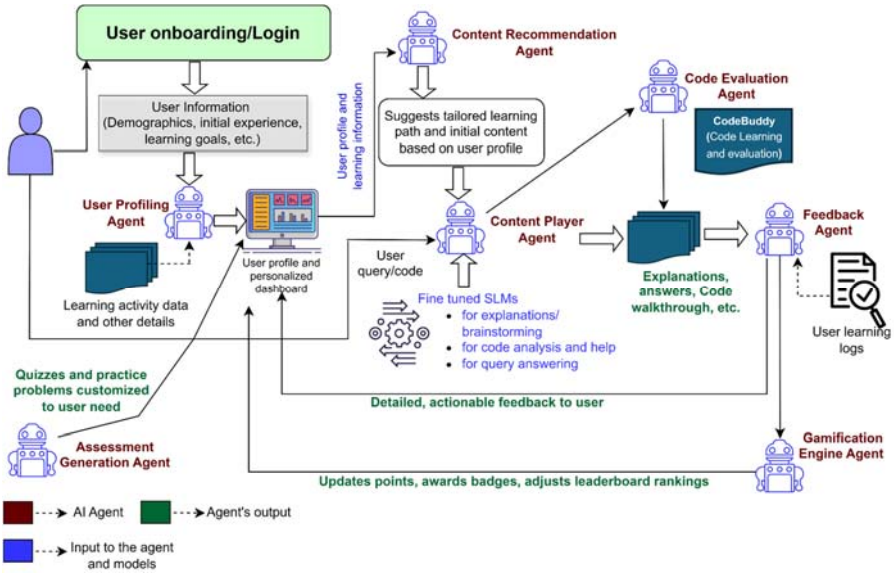


Fig. 2: Overview of the proposed aiTA framework

Step 2 (Content Recommendation Agent): Once the initial understanding of the user profile is established, the information is then fed into the *Content Recommendation Agent.* The *Content Recommendation Agent* is designed to guide learners by suggesting materials, tutorials, courses, documentation, or exercises that suit their skill level, learning style, and interests. The agent gathers data from the *User Profile and Personalised Dashboard,* such as lessons done, adopted problem-solving approach, time spent on each topic, and even error patterns. That profile is then the starting point from which programming languages, frameworks, or concepts that the learner is most likely to learn and gain from next are determined, customizing their learning path.

The *Content Recommendation Agent* uses methods like collaborative filtering and reinforcement learning to recommend customized learning paths and resources based on the individual user's specific progress and gaps in their knowledge. The process of recommending learning to program will involve integration between content-based and collaborative filtering methods. For instance, content-based filtering might recommend a Python tutorial to someone who has consistently interacted with data science-type material, looking for commonalities in terms of the programming language, topic (e.g., web development, machine learning), or difficulty level. Collaborative filtering, on the other hand, would take into account the fact that numerous students who finished a particular JavaScript course also finished a React framework course. On this basis, the algorithm would then recommend the React course to other students who demonstrate simultaneous advancement through the JavaScript course, essentially taking advantage of the aggregate learning paths of a large user base to recommend appropriate and helpful next steps.

Step 3 (Content Player Agent): The student can ask/submit their questions/code in the platform to seek solutions for their queries and code. Once the student submits his/her code, the *Content Player Agent* will analyse the query and be responsible for the active learning experience by analysing the user's learning patterns. The *Content Player Agent* is a fine-tuned language model (either SLM or MLM, or LLM, depending on the trained parameters, which is a point of research investigation) for natural language understanding, semantic analysis, and error detection for the programming language source code. The *Content Player Agent* facilitates real-time learning for the students by providing examples, explanations, generating brainstorming prompts, answering questions/queries, and writing code in any of the programming languages is heavily dependent on the language model. In such a context, we aim to investigate the use of co-learning among different language models (SLM, MLM, and LLM) to generate content, answer questions/queries, provide explanations, and code analysis from publicly available and augmented datasets. A separate pipeline can be created for each of the specialised tasks. The language model works not only by learning from a teacher model (State of the art LLM) but also through peer-learning, where models can help correct each other's predictions. Consequently, we develop an energy-efficient language model for the relevant classification-specific task.

Step 4 (Code Evaluation Agent): This module is for the users work with the source code (i.e., writing source code for the problem) and seeks help with the error and issue facing the source code, the *Code Evaluation Agent* evaluates the submitted source code for its correctness, effectiveness and adherence to the best practices. The code evaluation agent provides support in fixing code for hands-on coding support*, explaining code, and answering questions about the code* for conceptual understanding of the coding concepts. Each feature is designed to produce helpful responses while not directly generating code solutions. The code evaluation agent leverages code analysis techniques such as static code analysis, pre-trained code language models (e.g., PLBART), dynamic code execution by running test cases, and programming language pre-trained code language models for analyzing the code based on clarity, complexity, errors, warnings, and provides input to the *feedback agent* which provides helpful feedback for the questions asked by the student. The agent also provides analysis of the code, similar to the compiler and static analysis techniques, but in a detailed form that can be helpful for the students to identify the issues in the code. The agent also generates the metrics assessing the code quality

- **Cyclomatic Complexity**: Measures code complexity based on decision points.
- **Code Churn**: Tracks code changes over time, indicating stability and maintenance effort.
- **Code Coverage**: Assesses the percentage of code tested by automated tests.
- **Code Documentation**: Evaluates the presence and clarity of supporting documentation.
- **Code Duplication**: Identifies repeated code, reflecting adherence to the DRY principle and maintainability.
- **Bug Density**: Measures the number of defects per unit of code, indicating reliability.

Step 5 (*Feedback Agent*): The *Feedback Agent* translates findings of code evaluation and content player agents and sends rich, meaningful feedback to assist the learner in grasping errors and augmenting skills. The entire

process feeds back into the *User Profiling Agent*, which updates the user profile in real time. The feedback agent keeps track of the user learning logs and could be a fine-tuned SLM, similar to *Content Player Agent*, for analysing the user learning patterns and providing context-aware feedback on programming errors, conceptual misunderstandings, and areas of improvement instead of a correct/incorrect solution to the user. The user can check his/her dashboard for detailed and actionable feedback.

Step 6 (Gamification Agent): The feedback agent also sends the detailed feedback to the *Gamification Agent* to manage and update the gamification metrics, adjust rewards and leaderboards, further enhancing engagement. *The Gamification Agent* updates the points, awards badges, adjusts leaderboard rankings, and personalizes the gamified challenges to the user based on their feedback and learning patterns analysis. The gamification agent combines rule-based systems, event-driven programming, and reinforcement learning (RL) for optimizing engagement through gamification.

Step 7 (Assessment Generation Agent): The *Assessment Generation Agent,* along with the gamification agent, constantly produces adaptive quizzes and practice exercises related to learning goals and knowledge gaps. The agent leverages generative AI models and fine-tuned SLMs for the generation of quizzes, exercises, practice problems, and customised explanations based on the learning objectives. It is developed with sophisticated methodologies, specifically Generative AI (utilizing fine-tuned Small Language Models or SLMs), to generate a wide range of questions and scenarios. This enables the system to generate a variety of multiple-choice questions to complex problem-solving exercises, thus introducing fresh and relevant content with every iteration.

Step 8 (System Orchestrator Agent): This agent ensures seamless coordination among all components, enabling the system to continuously adapt to each learner's evolving needs through an ongoing feedback and refinement loop. As the learner progresses, the agents collaboratively update the educational experience, delivering highly responsive, personalized, and intelligent programming instruction.

Work Item-2: Evaluation of the aiTA proposed framework by conducting experimental analysis with the students, instructors, and human TAs.

To evaluate the efficacy, usability, effectiveness, and impact of the *aiTA* system on the students, instructors, and human TAs, we would adopt the quasi-experimental design. The quasi-experimental design will be considered to compare student groups using the proposed *aiTA* system with the control groups using the traditional TA support or without support. The design will determine the impact of the *aiTA* on learning performance, engagement, and learning behavior. Moreover, qualitative research will investigate user experience, challenges, and other aspects of the human-AI interaction. The following research questions will be used to perform the experimental analysis.

Research Questions (RQs):

RQ1: Does the use of an *aiTA* have a significant impact on students' academic performance compared to traditional Teaching Assistant (TA) support or no support?

RQ2: How does student engagement vary among groups using *aiTA*, those receiving traditional TA support, or no support?

RQ3: What measurable differences exist in student learning patterns among groups using *aiTA*, those receiving traditional TA support or no support?

The following RQs will evaluate user experience and human-AI interaction.

RQ4: What are students' perceptions about the *aiTA* system's effectiveness, usability, and overall value?

RQ5: What challenges did students and instructors encounter while interacting with the *aiTA*?

RQ6: How does the integration of the *aiTA* system into the teaching impact the quality of human-AI interaction in an educational environment?

RQ7: Does the use of the *aiTA* system facilitate or hinder students' independent learning and problem-solving approaches?

Experimental Subjects:

Students; Faculty/Instructors; Teaching Assistants (Humans)

**Informed consent will be obtained from all participants, and will follow the ethical guidelines stated by the Institutional Research Board (IRB).*

Evaluation Metrics:

- Student Engagement: Usage frequency of the framework, duration of interaction, and self-reported interest.
- Learning Outcomes: Continuous improvement on the assessment scores, understanding of complex concepts and achievement in the leaderboard, problem-solving abilities, and critical thinking.
- Efficiency and Workload: Can be measured by the time saved by the instructors/faculties/human TAs on administrative tasks (e.g., answering questions, solving doubts, fixing code) and the *aiTA*'s ability to provide timely support similar to human TAs.
- Framework Effectiveness: Adaptation of *aiTA* to individual student needs and learning styles, and provide relevant content and support.

Assessment of AI Model Performance[6],[7]

- Accuracy: How precisely the *aiTA* answers student queries and provides correct information on various tasks. The standard evaluation metrics, such as BLEU and ROUGE can be used[8].
- Relevance: Whether *aiTA* responses are pertinent to the student's query and the course context.
- Conversation Effectiveness: Evaluation metrics such as Long-term Coherence Tracking (LCT) and Cumulative Relevance Index (CRI) can be used.

Non-Functional Aspects and Ethical Considerations:

- Usability Testing: Assessments with smaller groups of users to identify the accessibility and interface challenges.
- Survey and Interviews: Detailed analysis of user interactions through their feedback and engagement patterns.
- Sentiment analysis: We can apply NLP techniques to analyse user feedback to detect their emotions, concerns and suggestions on the *aiTA* framework.
- Data Privacy and Security
- Impact on human interaction
- Academic Integrity: Over-reliance on the technological framework, User Pattern Analysis

Data Collection: To evaluate *aiTA's* performance, both quantitative and qualitative data will be collected. Quantitative measures are platform usage logs (e.g., response time, questions attempted), learning outcomes (quiz scores, grades, course completion), instructor workload reduction, and system performance (response time, error rates). In addition, qualitative feedback will be obtained through student and instructor surveys, interviews, and focus groups to measure usability, user experience, and teaching impact. In addition, *aiTA* will include an in-system feedback mechanism for real-time suggestions from users to enhance functionality and balance integration with human teaching aids.

Data Preparation: This process involves labelling or tagging (text annotation of lecture notes, textbooks, reading materials, examples, problem types, images, multimedia, etc.) of the content with descriptive information. Additionally, we would generate a comprehensive set of question-answer pairs based on the source code and the test cases. This dataset can be enhanced by augmentation techniques using the prompt-driven LLMs to create a wide range of pairs for training and analysis. We would also collect real-world queries from various sources, such as Stack Overflow and GitHub.

Data Analysis: We will adopt statistical analysis techniques (both descriptive and inferential) to identify patterns, correlations, and significant differences among groups (hypothesis testing) for analyzing the quantitative data.

Implementation Plan of the *aiTA* framework: We will follow a phase-wise iterative approach for the development of the aiTA framework. The development phases are:

- Phase I: Defining objective and scope - creation of software requirements specification (SRS) document.

---

[6]https://www.techrxiv.org/users/705459/articles/1213861-evaluating-conversational-ai-systems-for-responsible-integration-in-education-a-comprehensive-framework

[7] Gupta, Shailja, Rajesh Ranjan, and Surya Narayan Singh. "Comprehensive Framework for Evaluating Conversational AI Chatbots." arXiv preprint arXiv:2502.06105 (2025).

[8] https://precisiondev.org/evaluating-ai-for-learning-a-framework/

- Phase II: Identification of technology stack and design of the system architecture.
- Phase III: Development and training of the models (AI, ML, pre-trained language models) using prepared, annotated, and augmented datasets.
- Phase IV: Development of the user interface
- Phase V: Integration and Testing - Integration of aiTA with LMS platforms. Functional and non-functional testing of the system with students, instructors/faculties, and human TAs.
- Phase VI: Prototype Deployments and Analysis - Evaluation and assessment of the aiTA from different aspects.

We will adopt agile methodology for the development of the *aiTA* framework, specifically, using SCRUM. The development will be carried out in short cycles (sprints), which allows for frequent feedback and adjustments.

Work Item-3: Deployment of the System

The deployment of the proposed *aiTA* system would be modular, scalable, and cloud-native to support real-time responsiveness, high availability, and ease of integration. The front-end responsive UI/UX would be developed utilizing frameworks like React.js or Vue.js and would be hosted through platforms like Netlify or included in Kubernetes deployments behind an encrypted NGINX reverse proxy. Backend services, including course management, user profiling, gamification engines, and assessment modules would be developed in Python (FastAPI) or Node.js and integrated with databases such as PostgreSQL or MongoDB. AI models and agents will be deployed using model serving tools such as TorchServe, ONNX Runtime, or Triton Inference Server, with optimizations such as knowledge distillation, quantization, and pruning for real-time inference. Explanation and code assistance, Language Model agents would be integrated through OpenAI APIs or locally hosted fine-tuned models using LangChain. All services will be exposed securely via an API gateway with JWT and OAuth2 authentication. To provide security and meet India's DPDP Act requirements, data would be encrypted both at rest and in transit, role-based access would be enforced through tools such as Keycloak, and user privacy would be ensured through anonymization methods. Comprehensive documentation, feedback features, and monitoring dashboards would facilitate easy adoption, upkeep, and iterative refinement in institutions.

3.2 Suggested Plan of action for utilization of research outcome for social economic growth benefits expected from the project.

This project aims to harness the available sophisticated AI techniques to address diverse learner needs, align with curriculum standards, and overcome challenges of scale and inclusivity by leveraging natural language processing (NLP), code analysis, and recommendation systems to support students throughout their programming journey. Expected utilization of the outcomes includes:

1. Pilot Deployments in Diverse Educational Settings: We plan to conduct pilot deployments of the *aiTA* system, integrating it into existing programming courses within various educational areas. These deployments will enable real-world testing, gathering of critical user feedback, and iterative refinement of the system's efficacy in supporting student learning.

2. Startup Incubation and Ed-Tech Innovation: To foster broader adoption and innovation, we will open-source core *aiTA* modules under a permissive license, accompanied by comprehensive developer toolkits and deployment templates. We will actively pursue partnerships with incubators and Ed-Tech ventures to facilitate product development, explore commercialization pathways, and cultivate a vibrant ecosystem around *aiTA*.

3. Public-Private Partnerships for Infrastructure Support: To bridge critical digital-divide gaps and ensure equitable access to quality programming education, we will seek Public-Private Partnerships. These collaborations will facilitate the deployment of the *aiTA* system on scalable cloud infrastructure, ensuring its availability to a wide range of learners, including those in underserved communities.

4. Measuring Educational and Economic Impact: We will conduct a rigorous, evidence-based study quantifying the downstream effects of *aiTA's* implementation.

3.3 Environmental impact assessment and risk analysis

R1: IPR (Intellectual Property Rights) and privacy aspects when using GenAI on artifacts (e.g., source code) created by humans or using humans to generate artifacts using humans.

Mitigation: All data collection in India will be regulated by the ethical rules and IRB (Institutional Research Board) guidelines.

R2: Difficulties in recruiting students, Teaching Assistants (TAs), and faculties for experimentation and hands-on with the framework.

Mitigation: DAU and IIITM Gwalior have a large number of students at all levels, we can use them for the analysis as per the institutional rules and guidelines; we can collaborate with partner institutions and have MoU with them for data collection, experimentation and analysis of *aiTA*.

3.4 Dissemination plan: -

- Peer-Reviewed Publications: Publish the results and findings of the project to premier journals and flagship conferences.
- Open-Source Release: Release deployed agent modules, data-validation pipelines, and prompt-engineering templates on public repositories under an open-source license with detailed documentation.
- Patent Filings: Patent primary innovations, including the multi-agent workflow and active-learning QA mechanisms.
- Workshops & Seminars: Conduct hands-on workshops and seminars to educate faculties, educational institutes, and IT professionals about the developed system.
- Industry Engagement: Engage technology partners through pilot deployments and shared webinars, highlighting use-case stories, and performance metrics.

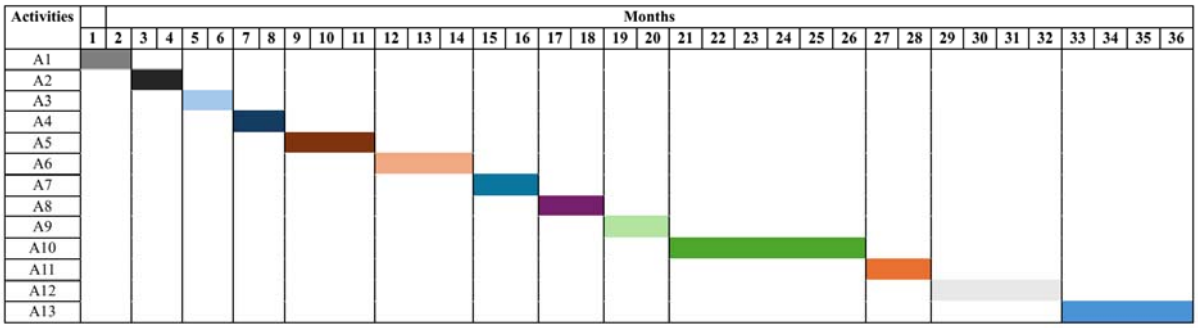
3.5 Time Schedule of activities giving milestones through BAR diagram.

The 36 months' project is divided into five milestones. Intermediate objectives/work elements are defined for each milestone. Table 1 highlights the work elements planned for each of the milestones.

Table 1: Work Elements of milestones

S. No.	Activity	Duration	Timeline (Months)	Outcomes of each activity
1	Requirement Analysis, Literature Review, and Use-case Finalization	2 months	Month 1–2	Understand the virtual TA landscape and finalize system requirements.
2	Architecture Design and Agent Workflow Blueprint	2 months	Month 3–4	Multi-agent orchestration and modular component design
3	Development of User Profiling Agent	2 months	Month 5–6	Learner modeling and profile tracking
4	Development of Content Recommendation Agent	2 months	Month 7–8	Real-time curriculum adaptation
5	Development of Interactive Content Player Agent with SLM Integration	3 months	Month 9–11	NLP interface, debugging, and feedback support
6	Development of Code Evaluation and Semantic Feedback Agents	3 months	Month 12–14	Syntax analysis, static/dynamic evaluation
7	Development of Gamification Engine & Discussion Forum Module	2 months	Month 15–16	Engagement: badges, points, leaderboard
8	Development of Assessment Generation Agent	2 months	Month 17–18	Dynamic quiz/task generation via GenAI
9	Integration & Development of System Orchestration Agent	2 months	Month 19–20	Coordination of multi-agent system
10	• Frontend Development and Dashboard Design • Backend Application Layer Setup • Internal Testing, Debugging, and Refinement of Modules	6 months	Month 21–26	Responsive, intuitive interface for learners & instructors
11	Pilot Deployment at some Indian Institutions	2 months	Month 27–28	Feedback-driven improvements
12	Controlled Field Evaluation with Teachers, Students, Human TAs	4 months	Month 29–32	IRB-compliant mixed-method study
13	Documentation (User manuals, admin guide, tech spec), Dissemination: Publications, Conferences, Open Source Release Technology Transfer, IP Filing (if any), and Final Reporting	4 months	Month 33-36	

A1: Requirement Analysis, Literature Review, and Use-case Finalization. A2: Architecture Design and Agent Workflow Blueprint, A3: Development of User Profiling Agent, A4: Development of Content Recommendation Agent, A5: Development of Interactive Content Player Agent with SLM Integration, A6: Development of Code Evaluation and Semantic Feedback Agents, A7: Development of Gamification Engine & Discussion Forum Module, A8: Development of Assessment Generation Agent, A9: Integration & Development of System Orchestration Agent, A10: Frontend, Backend Development, Dashboard Design, and Internal Testing, A11: Pilot Deployment at some Indian Institutions, A12: Controlled Field Evaluation with Teachers, Students, Human Tas, A13: Documentation, Dissemination, Final Reporting.



3.6 Bibliography

[1] S. Hobert and F. Berens, "Developing a digital tutor as an intermediary between students, teaching assistants, and lecturers," Educational Technology Research and Development, vol. 72, no. 2, pp. 797–818, 2024. [Online]. Available: https://doi.org/10.1007/s11423-023-10293-2

[2] J. Lui, B. Jiang, and Y. Wei, "LLMs as promising personalized teaching assistants: How do they ease teaching work?" ECNU Review of Education, vol. 8, no. 2, pp. 343–348, 2025. [Online]. Available: https://doi.org/10.1177/20965311241305138

[3] T. Feng, S. Liu, and D. Ghosal, "Courseassist: Pedagogically appropriate ai tutor for computer science education," in Proceedings of the 2024 on ACM Virtual Global Computing Education Conference V. 2, ser. SIGCSE Virtual 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 310–311. [Online]. Available: https://doi.org/10.1145/3649409.3691094

[4] T. Wang, Y. Zhan, J. Lian, Z. Hu, N. J. Yuan, Q. Zhang, X. Xie, and H. Xiong, "Llm-powered multi-agent framework for goal-oriented learning in intelligent tutoring system," 2025. [Online]. Available: https://arxiv.org/abs/2501.15749

[5] J. Tahir, R. Mandal, O. Stefanova, H.-A. Jacobsen, C. Doblander, and R. Mayer, "pta: A programmable teaching assistant for lab courses," ser. DataEd '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 24–29. [Online]. Available: https://doi.org/10.1145/3596673.3596975

[6] M. Wu, N. Goodman, C. Piech, and C. Finn, "Prototransformer: A meta-learning approach to providing student feedback," 2021. [Online]. Available: https://arxiv.org/abs/2107.14035

[7] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, "Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs," in Proceedings of the CHI Conference on Human Factors in Computing Systems, ser. CHI '24. ACM, May 2024, p. 1–20. [Online]. Available: http://dx.doi.org/10.1145/3613904.3642773

[8] C. Lee, J. Myung, J. Han, J. Jin, and A. Oh, "Learning from teaching assistants to program with subgoals: Exploring the potential for AI teaching assistants," 2023. [Online]. Available: https://arxiv.org/abs/2309.10419

[9] M. Liffiton, B. E Sheese, J. Savelka, and P. Denny. "CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes". In Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23). Association for Computing Machinery, Article 8, 1–11, 2024. [Online]. https://doi.org/10.1145/3631802.3631830

4. Expertise:

4.1 Expertise available with the investigators in executing the project:

- The PI (Dr. Saurabh Tiwari) has completed his doctoral degree in Experimental Software Engineering (Thesis Title: "Evaluating Usability Aspects of Use Cases for Software Specification Problems"). His research interests are in the area of Requirements Engineering, Empirical Software Engineering, Mining Software Repositories, Green Computing, GenAI techniques, and NLP. He is also doing research in Software Engineering Education and Training to identify/devise innovative teaching methodologies to teach Software Engineering concepts. He has also developed various tools as a part of his research work (such as UCAnalyser, Text2UseCase, UC2Map, Inclusivity Checker, AutoReco, CORMS, GW2UPPAAL, and NL2RT). He has published several research papers in various international conferences (ESEC/FSE, RE, ICSE, APSEC, ISEC, ICECCS, SAC, etc.) and journals (IST, CS&I, JSEP, CAE, etc). He has also received two best paper awards at the 20th APSEC 2013 and 25th APSEC 2018 conferences, respectively, and his PhD thesis is indexed in the ACM (Association for Computing Machinery) SIGSOFT (Special Interest Group on Software Engineering) list of PhD Dissertations in the area of Software Engineering

(2016). The PI has completed two sponsored projects in the area of Software Engineering and Security sponsored by the SERB and DRDO.

- The Co-PI (Dr. Santosh Singh Rathore) has completed his Doctorate degree in Machine Learning and Software Engineering (Thesis Title "Predicting Number of Faults in Software Systems"). Specifically, the PhD thesis is in predictive modeling, which deals with the development of accurate and effective prediction models, and efficient knowledge representation. His research interests include Machine learning, knowledge representation, teaching-learning, and educational research. He has authored over 50 peer-reviewed publications in predictive modeling, knowledge representation, and teaching pedagogy, and has authored 2 books on predictive modeling. The Co-PI also completed 4 sponsored projects in the area of Computer Science. His knowledge in AI models, teaching pedagogies, and Generative AI (GenAI) will underpin the development of different AI agents.
- The Co-PI (Dr. Sourish Dasgupta) is an ever-curious researcher and educator. His deep interest in the role of AI in research methodologies led him to take a break from academia and set on an entrepreneurial journey for five years, which resulted in the founding of RAx Labs Inc., Delaware, USA. With some bright ex-students of DA-IICT, he built RAx (https://raxter.io) - an AI-powered online assistant for making literature-review faster and more enriching for young researchers. Coming back from his entrepreneurial stint, he is currently actively engaged in the analysis of less explored but important aspects of "intelligence" in LLMs, such as their personalization capabilities, and also designing personalized models that are a lot smaller and eco-friendlier than contemporary LLMs. Sourish did his Ph.D. in Computer Science from the University of Missouri – Kansas City, USA.

4.2 Summary of roles/responsibilities for all Investigators:

S. No.	Name of the investigators	Roles/ Responsibilities	Specific part of the proposals where PI/Co-PI Contribution
1.	Saurabh Tiwari	Principal Investigator	Software Engineering, Software Development, NLP (focused on software development, data annotations, labeling, model training, development of agentic AI framework)
2.	Santosh Singh Rathore	Co-Principal Investigator	Software Engineering, Machine Learning, Deep learning, AI (focused on model generation, model fine-tuning, distilled language models, development of agentic AI framework)
3.	Sourish Dasgupta	Co-Principal Investigator	AI, NLP and Personalisation (focused on the development of energy-efficient language models through knowledge distillation and transfer learning techniques, development of Agentic AI framework, and personalisation)

4.3 Key publications published by the Investigators pertaining to the theme of the proposal during the last 5 years

1. Prutha Patani, Saurabh Tiwari, and Santosh Singh Rathore, "The Impact of GitHub on Students' Learning and Engagement in Software Engineering Course", Computer Applications in Engineering Education, Volume 32, Issue 5, Pages 1-23, September 2024, e22775, https://doi.org/10.1002/cae.22775
2. Saurabh Tiwari and Santosh Singh Rathore, "Understanding General Concepts of Requirements Engineering through Design Thinking: An Experimental Study with Students", Computer Applications in Engineering Education, Volume 30, Issue 6, November 2022, Pages 1683-1700, https://doi.org/10.1002/cae.22550
3. Saurabh Tiwari and Santosh Singh Rathore, Leveraging LLMs for Requirements Engineering Education: How to Approach?, In 33rd IEEE International Requirements Engineering conference (RE 2025), RE@Next, Valencia, Spain, September 1-5, 2025 [Accepted]
4. Shivani Balwani, Saurabh Tiwari, Sourish Dasgupta, and Akhilesh Sharma, AutoReco: A Tool for Recommending Requirements for their Non-Conformance with Requirement Templates (RTs), In 31st IEEE International Requirements Engineering 2023 conference (RE 2023), Tool Demonstration Track, Leibniz Universität Hannover, Germany, September 4-8, 2023.
5. Kushagra Pathak, Parita Patel, Mital Kamani and Saurabh Tiwari. Inclusivity Checker: A Testing Tool to Detect Inclusivity Bugs in Websites. In 16th Innovations in Software Engineering Conference (ISEC 2023), New York,

NY, USA, Article 23, 1–5, 23-25 February 2023, Research Track, IIIT Allahabad, https://doi.org/10.1145/3578527.3578547.

6. Saurabh Tiwari, Santosh Singh Rathore, Sheikh Umar Farooq, and Prutha Patani. How Students Are Using GitHub? A Survey. In 15th Innovations in Software Engineering Conference (ISEC 2022), 24-26 February, Association for Computing Machinery (ACM), Article 26, pages 1–5, 2022.
7. Deepti Ameta and Saurabh Tiwari, Why does the experiment fail with students? An Experience, In 14th Innovations in Software Engineering Conference (ISEC 2021), 25-27 February, ACM, Article 15, pages 1–5, 2021.
8. Saurabh Tiwari, Rishab Arora, and Ashray Bharambe, UC2Map: Automatic Translation of Use Case Maps from Specification, In 35th ACM-SIGAPP Symposium on Applied Computing, SAC 2020, pp 1650-1653, Brno, the Czech Republic, March 30-April 3, 2020.
9. A Mangal and Santosh Singh Rathore, "ATE-FS: An Average Treatment Effect-based Feature Selection Technique for Software Fault Prediction," In ACM Transactions on Intelligent Systems and Technology, vol. 16, issue 3, pp. 1-28, 2025.
10. Monika Choudhary, Satyendra Singh Chouhan, and Santosh Singh Rathore, "Beyond Text: Multimodal Credibility Assessment Approaches for Online User-Generated Content," In ACM Transactions on Intelligent Systems and Technology, vol. 15, issue 5, article no. 92, pp. 1 - 33, 2024.
11. Aman Omar, Santosh Singh Rathore, and Sandeep Kumar, "ME-SFP: A Mixture of Experts based Approach for Software Fault Prediction," In IEEE Transactions on Reliability, vol. 73, issue 1, pp. 710-725, IEEE, 2023.
12. Swarup Padhy and Santosh Singh Rathore, "An attention-based deep learning model for credibility assessment of online health information," In Computational Intelligence, Wiley, vol. 39, issue 5, pp. 832-859, 2023.
13. Aditya Shankar Mishra and Santosh Singh Rathore, "Implicit and explicit mixture of experts models for software defect prediction," In Software Quality Journal, vol. 31, pp. 1331-1368, Springer, 2023.
14. Lalit Purohit, Santosh S. Rathore, and Sandeep Kumar, "A QoS-Aware Clustering based Multi-layer Model for Web Service Selection," In IEEE Transactions on Services Computing, vol. 16, issue 5, pp. 3141- 3154, IEEE, 2023.
15. Jitendra Parmar, Satyendra Singh Chouhan, Vaskar Raychoudhury, and Santosh S. Rathore, "Open-world Machine Learning: Applications, Challenges, and Opportunities," In ACM Computing Survey, vol. 55, issue 10, article no. 205, pp. 1 - 37, 2022.
16. Sanskar Soni, Satyendra S. Chouhan, and Santosh S. Rathore, "TextConvoNet: a convolutional neural network-based architecture for text classification," In Applied Intelligence, vol. 53, pp. 14249–14268, 2022.
17. Santosh Singh Rathore, Satyendra Singh Chouhan, Dixit Kumar Jain, and Aakash Gopal Vachhani, "Generative Oversampling Methods for Handling Imbalanced Data in Software Fault Prediction," In IEEE Transactions on Reliability, vol. 71, No. 2, 2022.
18. Santosh Singh Rathore, "An exploratory analysis of regression methods for predicting faults in software systems," In Soft Computing, Springer, vol. 25, pp. 14841–14872, 2021.
19. Santosh Singh Rathore and Sandeep Kumar, "Software Fault Prediction based on the Dynamic Selection of Learning Technique: Findings from the Eclipse Project Study," In Applied Intelligence, vol. 51, pp. 8945–8960, 2021.
20. Satyendra S. Chouhan and Santosh S. Rathore, "Generative Adversarial Networks-Based Imbalance Learning in Software Aging-Related Bug Prediction", In IEEE Transactions on Reliability, 2021, vol. 99, pp. 1-19.
21. Sourish Dasgupta, Harsh Sharma, Devansh Patel, Prarthee Desai, & Anil K Roy, "Are Key-Phrases All That Reviewers Care About? A Comprehensive Benchmarking of Reviewer Matchmaking Systems". Proceedings of the AAAI Conference on Artificial Intelligence, 39(22), 23743-23751, 2025
22. Sourish Dasgupta, Ankush Chander, Parth Borad, Isha Motiyani, and Tanmoy Chakraborty. "PerSEval: Assessing Personalization in Text Summarizers." arXiv preprint arXiv:2407.00453 (2024).
23. Divya Patel, Pathik Patel, Ankush Chander, Sourish Dasgupta, and Tanmoy Chakraborty. "Are Large Language Models In-Context Personalized Summarizers? Get an iCOPERNICUS Test Done!." arXiv preprint arXiv:2410.00149 (2024).
24. Rahul Vansh, Darsh Rank, Sourish Dasgupta, and Tanmoy Chakraborty. 2023. Accuracy is not enough: Evaluating Personalization in Summarizers. In Findings of the Association for Computational Linguistics: EMNLP 2023, pages 2582–2595, Singapore. Association for Computational Linguistics.
25. Priyanshi Gupta, Yash Kumar Atri, Apurva Nagvenkar, Sourish Dasgupta, and Tanmoy Chakraborty. "Inline citation classification using peripheral context and time-evolving augmentation." In Pacific-Asia Conference on Knowledge Discovery and Data Mining, pp. 3-14. Cham: Springer Nature Switzerland, 2023.

4.4 List of funded projects implemented/ being implemented:

(All the Investigators should list details of the projects submitted, implemented, and completed by them under various funding agencies in the table below. The list should start with the Projects implemented by the Principal Investigator, followed by Co-PI 1, Co-PI 2, Investigators, etc.)

S.No.	Project Title	Funding Agency	Budget	Duration	Start date & End Date (MM/YYYY)	Status	Role as PI/ CoPI	PCR evaluation grade it funded by (ANRF/ SERB)	Summary of Result & Publication/ Patent
1	An Empirical Analysis on Deriving Test Cases from Natural Language Text using MBT Approach	SAC-ISRO	20.78 Lac	3 Years	May 2022	Ongoing	PI	-	2 Research Publications in RE 2023 and ISEC 2024 conference
2	Vulnerability Research on QUIC Implementations	DRDO	22.66 Lac	1.5 Years	Nov 2022	Completed	Co-PI	-	1. A prototype is developed. 2. Performed experimental analysis on QUIC protocol
3	UCMA: A Toolset to Automatically Analyse Functional Requirements Specified in the Use Cases	SERB	9.65 Lac	2 Years	March 2017	Completed	PI	Good/SERB	7 Research Papers Publications in: 1. 25th RE 2017 2. 12th ISEC 2019 (#2 Publications) 3. Information and Software Technology Journal 4. 35th SAC 2020; 5. 28th RE 2020; 6. Journal of Software - Evolution and Process
4	Artificial Intelligence Enabled Dialogue- based Online Medical System Development for Precision Healthcare	SERB, DST	15.43 Lac	02 years	01/2022 to 01/2024	Completed	PI	SERB	3 Research Publications in SCI Journals
5	AUTOEMR: An AI-based System for Generating Structured Medical Information	TIIC, ABV-IIITM Gwalior	3.0 Lac	01 Year	11/2021 to 12/2022	Competed	PI		A prototype is developed.
6	Design of NextGen Hearing Aids	TIIC, ABV-IIITM Gwalior	3.0 Lac	01 Year	11/2021 to 12/2022	Competed	Co-PI		A prototype is developed.
7	Inferring High-level Code Similarities in Software Systems using String-Metric based Hybrid Technique	TEQIP III, NIT Jalandhar	3.0 Lac	01 Year	05/2018 to 06/2019	Completed	Co-PI		N/A
8	An Empirical Analysis on Deriving Test Cases from Natural Language Text using MBT Approach	SAC-ISRO	20.78 Lac	3 Years	May 2022	Ongoing	Co-PI	-	2 Research Publications in RE 2023 and ISEC 2024 conference

5. List of facilities being extended by parent institution(s) for the project implementation.

5.1 Infrastructural Facilities

S. No.	Infrastructural Facility	Yes/No/ Not required Full or sharing basis
1.	Workshop Facility	Not required Full or sharing basis
2.	Water & Electricity	Yes (Full)
3.	Laboratory Space/Furniture	Yes (Full)
4.	Power Generator	Yes (Shared)
5.	AC Room or AC	Yes (Full)
6.	Telecommunication including email & fax	Yes
7.	Transportation	Yes
8.	Administrative/ Secretarial support	Yes

9.	Information facilities like Internet/ Library	Yes (Full)
10.	Computational facilities	Yes (Partial)
11.	Animal/ Glass House	Not required
12.	Any other special facility being provided	Not required Full or sharing basis

5.2 Equipment available with the Institute/ Group/ Department/ Other Institutes for the project:

Equipment available with	Specific Name of Equipment & detailed description	Model, Make & year of purchase	Remarks including accessories available and current usage of equipment
Co-PI & his group	1 Laptop, 1 Personal Computer, & 1 Desktop PC with GPU	Laptop: HP, Purchased in 2019 Personal Computer: Dell, Purchased in 2020 Desktop PC with GPU: Tyron, Purchased in 2022	Laptop: 100%, Personal Computer: 80% Desktop PC with GPU: 60%
Co-PI's Department	N/A	N/A	N/A
Other Institute(s) in the region	N/A	N/A	N/A

Equipment available with	Specific Name of Equipment & detailed description	Model, Make & year of purchase	Remarks including accessories available and current usage of equipment
PI & his group	1 Laptop, 1 Personal Computer, 1 Desktop PC. 1 Printer, and 1 Desktop PC with GPU	Laptop: Dell, Purchased in 2020 Personal Computer: Apple, Purchased in 2018 Desktop PC: Intel, purchased in 2020, Desktop PC with GPU: Intel purchased in 2022	Laptop: 100%, Personal Computer: 80%, 1 Desktop PC: 80%, Desktop PC with GPU: 50%
PI's Department	N/A	N/A	N/A
Other Institute(s) in the region	N/A	N/A	N/A

6. Name and address of experts/ institutions interested in the subject/ outcome of the project.

S. No.	Name of Expert Affiliation	Email	Area of Expertise
1	Prof Amey Karkare, IIT Kanpur	karkare@iitk.ac.in	Programming Languages and Education
2	Prof Rushikesh Joshi, IIT Bombay	rkj@cse.iitb.ac.in	Computational Models for Programs, their Structures and Dynamics, Software Engineering
3	Prof Pankaj Jalote	jalote@iiitd.ac.in	Software quality, process improvement;,Higher Education in India

7. Current financial support, other than those mentioned in section 4.4.

N/A

8. Any other relevant information & supplementary documentation.

N/A